

Mikkey

A keychain-sized AI buddy who physically cannot speak. He only sings. That was a design decision and I regret it.

Mikkey hangs off your keys. His screen is his mouth, animated in sync with his own audio so he looks like he's forming the words. He has a microphone, so he hears you — ask him something in English, Russian, Kazakh or Mandarin and he sings the answer back in the language you used. He wears a small 3D-printed hat. It does nothing. It stays.

This guide takes you from nothing to a singing keychain in about half an hour. You need one off-the-shelf board and one API key. Everything else is in the repo.

START HERE

github.com/jinglebells2020/mikkey

```
git clone https://github.com/jinglebells2020/mikkey.git
cd mikkey
```

1 What you need

THING	NOTES
M5StickS3	The whole build. ESP32-S3-PICO-1-N8R8, 8 MB flash + 8 MB PSRAM, 1.14" LCD, ES8311 audio codec with a MEMS microphone and AW8737 amplifier, 6-axis IMU, 250 mAh battery — screen, mic, speaker and power already inside, nothing to wire. shop.m5stack.com/products/m5sticks3-esp32s3-mini-iot-dev-kit
A laptop	macOS in this guide (the helper scripts are bash and use <code>ipconfig</code> / <code>scutil</code>). The Python side is portable; the two shell scripts need small edits on Linux.
Fish Audio API key	Required. This is his voice and his entire personality. Sign up at fish.audio and make a key — the guide uses the S2.1 Pro model.
OpenRouter API key	Optional. This is his brain, for answering questions he hears. Without it he still sings everything you type and every scripted line; he just can't improvise.
A 3D-printed hat	Entirely optional. Does nothing. Recommended.

2 How it works

The stick is the body; your laptop is the brain. They talk over USB serial, or over TCP once the stick joins WiFi. **The device is the authority over the link** — it announces itself with `POLL` on whichever transport it is reading, and the

laptop only pushes audio in reply.

```
[ M5StickS3 ] <---- USB serial OR TCP over WiFi ----> [ laptop ]
screen / mic                                     mikkey_server.py
speaker / button                                :8090 http   :8091 tcp

hold the button --> mic audio --> faster-whisper --> OpenRouter (in character)
                                                         |
speaker + mouth <-- PCM + envelope <-- Fish Audio S2.1 <-----+>
```

PIECE	WHAT IT DOES
Fish Audio S2.1 Pro	Sings every line. You direct the delivery in square brackets, like a session singer: <code>[breathy, sing-song, like a lullaby]</code> . Same words, different character. This is why Mikkey exists — with a flat voice there is no Mikkey.
faster-whisper	His ears, running locally on the laptop. Auto-detects the language among an allowlist so short noisy clips don't wander off into Finnish.
OpenRouter	His brain. One short sung line per answer, in the language you spoke, with an English direction tag.
Mouth-sync envelope	The laptop computes a loudness envelope alongside the audio and ships both. The face opens and closes on the envelope, so he looks like he's forming the words.
M5Unified	The Arduino library behind the screen, speaker, mic, IMU and button.

3 Build it

1 Python side

```
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
```

Pulls in the Fish Audio SDK, faster-whisper, numpy, pyserial and sounddevice. The first run downloads the whisper model (a few hundred MB), once.

2 Keys

```
cp .env.example .env
# then edit .env:
FISH_API_KEY=sk-fish-your-key-here
OPENROUTER_API_KEY=sk-or-your-key-here # optional, the brain
```

`.env` is gitignored. Keep it that way — never commit it.

3 Arduino toolchain

```
brew install arduino-cli
arduino-cli core update-index --additional-urls \
  https://raw.githubusercontent.com/espressif/arduino-esp32/gh-
  pages/package_esp32_index.json
arduino-cli core install esp32:esp32 --additional-urls \
  https://raw.githubusercontent.com/espressif/arduino-esp32/gh-
  pages/package_esp32_index.json
arduino-cli lib install M5Unified
```

Known-good here: arduino-cli 1.4.1, esp32 core 3.3.8, M5Unified 0.2.20, M5GFX 0.2.27.

4 Tell him where the laptop is

Plug the stick in over USB, then:

```
./set_wifi.sh "your-network" "your-password"
```

That writes `firmware/mikkey/wifi_config.h` (gitignored — it holds your WiFi password) with your laptop's mDNS name and current IP, then compiles and flashes in one go. USB alone is enough to get him singing; WiFi is what frees him from the cable.

5 Flash by hand, if you'd rather

```
FQBN="esp32:esp32:esp32s3:CDC0nBoot=cdc,FlashSize=8M,PSRAM=opi,PartitionScheme=default_8MB"
arduino-cli compile --fqbn "$FQBN" firmware/mikkey
arduino-cli upload --fqbn "$FQBN" -p /dev/cu.usbmodem* firmware/mikkey
```

Those flags matter: `CDC0nBoot=cdc` gives you the USB serial link, and `PSRAM=opi` is what lets him hold a whole clip in memory before playing it.

6 Start the brain

```
./run.sh
```

Preflight checks, then the server with auto-restart and timestamped logs. It pre-generates every line in `script.yaml` at startup, so they play instantly later. Leave it running.

7 Make him sing

In a second terminal:

```
./venv/bin/python mikkey_type.py

mikkey> Hello, I am Mikkey, and I live on your keys.
```

That's it. A blue dot in the top-left corner of his screen means the TCP link is up; grey means he's talking over USB instead. Both work.

4 Living with him

DO THIS	HE DOES THIS
Tap the button	Sings the next line of <code>script.yaml</code> , in order.
Hold the button	Push-to-talk. Speak, release, and he sings an answer back in the language you used.
<code>mikkey_type.py</code>	Type a line, he sings it. <code>/dir <words></code> sets the standing direction; a typed <code>[tag]</code> overrides it for one line. <code>/ls</code> , <code>/l <n id></code> , <code>/next</code> and <code>/goto</code> drive the script from the keyboard.
<code>localhost:8090/studio</code>	The same thing in a browser: two fields, <i>direction</i> and <i>line</i> . Change only the direction and sing again — that contrast is the whole demo.
<code>localhost:8090/preview</code>	His face on your screen, so you can see the animation without squinting at a 1.14" LCD.
Look in <code>out/</code>	Every line he has ever sung, as an audio master. <code>out/takes.log</code> records what played, when, and which file it is.

5 Make him yours

TO CHANGE	EDIT
What he says	<code>script.yaml</code> . Each line has an id and text; changing the text regenerates the audio on the next server start.
How he says it	The bracket tag at the front of any line. <code>[sung like a stadium rock anthem, powerful, shouty]</code> and <code>[breathy, sing-song, like a lullaby]</code> on the same words is the demo worth doing first.
Who he is	The <code>SYSTEM</code> prompt in <code>mikkey_server.py</code> — his character, the one-short-line rule, and the rule that he answers in the language he was spoken to.
His face	<code>FACE_STYLE</code> in <code>firmware/mikkey/mikkey.ino</code> : 2 = 1-bit pop-star line art, 1 = a full-screen grid face, 0 = landscape scenes. The line art is ASCII in <code>tools/mj_art.py</code> — run it to regenerate the header and the <code>/mj</code> preview. <code>FACE_ROTATION</code> picks which end is up, for your hat.
His voice	<code>model</code> in <code>script.yaml</code> . Pin a specific voice with <code>reference_id</code> .
Languages he'll accept	<code>MIKKEY_LANGS</code> (default <code>en,ru,kk,zh</code>). An allowlist beats free auto-detection on short, noisy clips.

6 Things that will otherwise cost you an hour

WIFI — THE ONE THAT REALLY BITES

Home routers with band steering or a weak 2.4 GHz radio will simply refuse the stick. The reliable fallback is a phone hotspot with **Maximize Compatibility** turned on. The stick prints `dbg wifi ...` on USB serial: reason 2 or 4 means the access point never answered, 15 means the WPA2 handshake timed out. USB works regardless, so a stubborn network never blocks you.

USB SERIAL — DO NOT REGRESS THIS

Host writes must be paced at roughly 250 kB/s or the USB CDC link stalls permanently, and the receive ring silently drops bytes if you flood it while the firmware is drawing. The server already paces streams at about 1.4× real-time. If you rewrite the transport, keep the pacing.

BATTERY

250 mAh is under an hour of singing. For a long session, run him off a USB power bank.

KAZAKH

Whisper detects Russian and Mandarin reliably but hears Kazakh as Turkish, at both `base` and `small`. Pin the language on `/studio` before a Kazakh exchange, then set it back to auto.

7 **When something is wrong**

SYMPTOM	CAUSE AND FIX
"mikkey_server.py is not running"	Start <code>./run.sh</code> first. The terminal and the browser tools are both clients of it.
Server won't start	Missing <code>FISH_API_KEY</code> in <code>.env</code> , or no <code>.venv</code> . The preflight in <code>run.sh</code> names which.
Nothing on USB	No <code>/dev/cu.usbmodem*</code> — reseal the cable, and use a data cable, not a charge-only one.
He's silent but the screen works	Look at the server log. Generation failures print a reason; a failed line is requeued rather than skipped.
A line sounds wrong	Change only the bracket tag and sing it again. Keep the words and punctuation identical — an added ellipsis rephrases the whole line.
Where's the audio?	<code>out/<id>.mp3</code> or <code>.wav</code> , indexed by <code>out/takes.log</code> .

Everything he can fail at, he fails out loud: a broken brain or a dead TTS call becomes a sung apology, a dropped link requeues the clip and replays it, and errors put a face and a reason on the screen. He is never silently broken.